哈爾濱工業大學
HARBIN INSTITUTE OF TECHNOLOGY

# Lab 3：代码评审与单元测试

哈爾濱工業大學
HARBIN INSTITUTE OF TECHNOLOGY

# 代码评审

# 实验要求

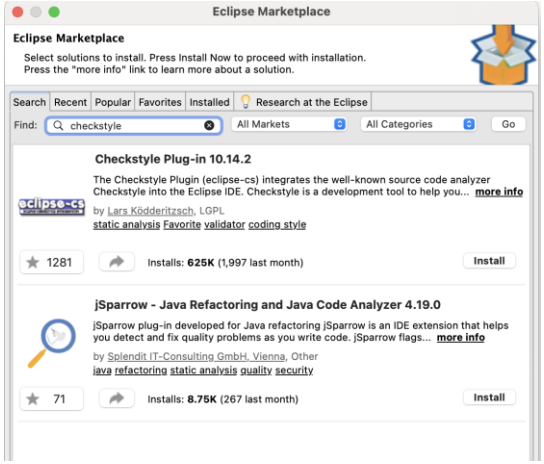
- **针对Lab1所完成的代码，进行代码评审(走查)，从代码规范性和正确性角度对代码进行评价；**
- **使用以下两个工具完成实验：**
  - Checkstyle
  - SpotBugs
- **注：如果使用Python等其他语言，自行选取对应的工具完成实验，例如Pylint、Ruff等。**

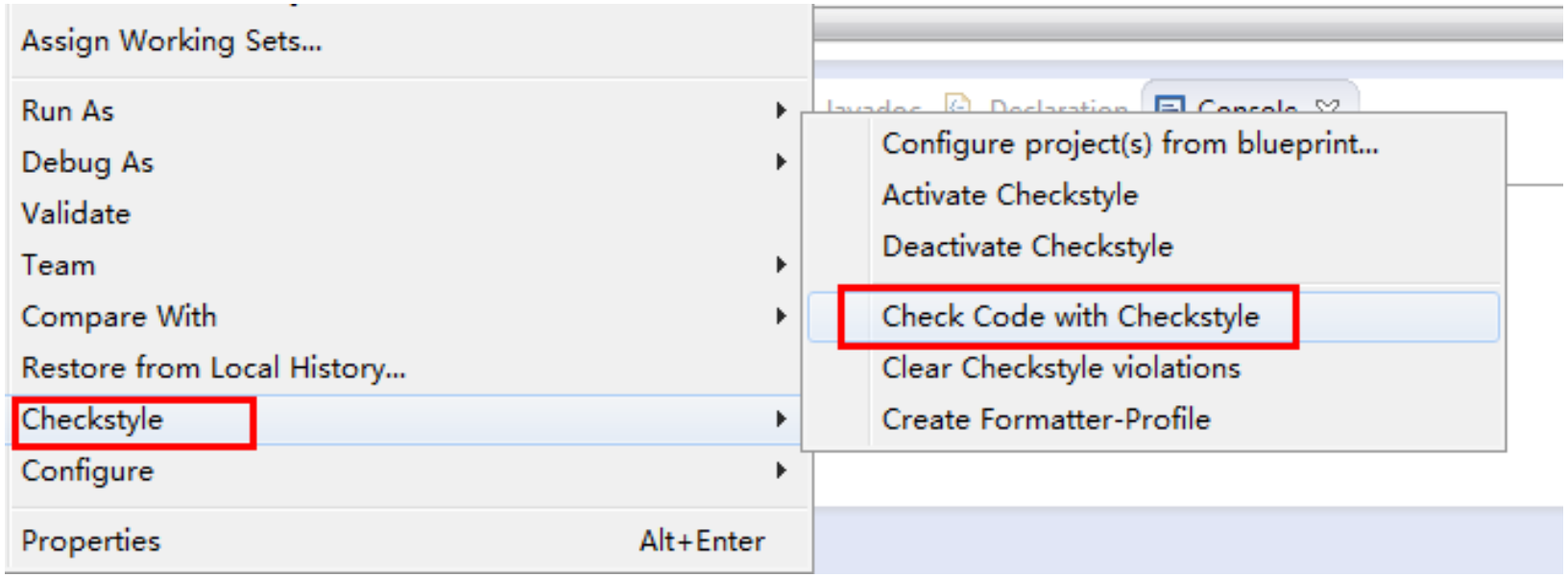
# Checkstyle

- **CheckStyle帮助Java开发人员遵守某些编码规范，能自动化代码规范检查过程，从而使开发人员从这项重要但是枯燥的任务中解脱出来。请从官网上阅读详细使用手册：https://checkstyle.org/index.html**
- **CheckStyle检验的主要内容：**
  - Javadoc注释
  - 命名约定
  - 标题
  - Import语句
  - 体积大小
  - 空白
  - 修饰符
  - 块
  - 代码问题
  - 类设计

# Checkstyle in Eclipse

- **Eclipse下的安装和使用**
  - 从Eclipse MarketPlace中搜索CheckStyle安装
  - 安装成功后，选中工程，右键选择Checkstyle->check code with checkstyle，检查错误即可。



# Checkstyle in Eclipse

- 检查结果在每行开始显示图标，鼠标移动至其上时显示详细问题；

Java - Checker.java -

File Edit Source Refactor Navigate Search Project Run Window Help

Java | Resource

DefaultContext.java | CheckStyleTask.java | DefaultConfigurat... | Checker.java

```
/**
 * This class provides the functionality to check a set of files.
 * @author Oliver Burn
 * @author <a href="mailto:stephane.bailliez@wanadoo.fr">Stephane Bailliez</a>
 * @author lkuehne
 */
public class Checker extends AutomaticBean
    implements MessageDispatcher
{
    /** maintains error count */
    private final S[First sentence should end with a period.] nter =
            new Se[Press 'F2' for focus.] tyLevel.ERROR);

    /** vector of listeners */
    private final ArrayList mListeners = new ArrayList();

    /** vector of fileset checks */
    private final ArrayList mFileSetChecks = new ArrayList();

    /** class loader to resolve classes with. **/
    private ClassLoader mLoader =
            Thread.currentThread().getContextClassLoader();
```

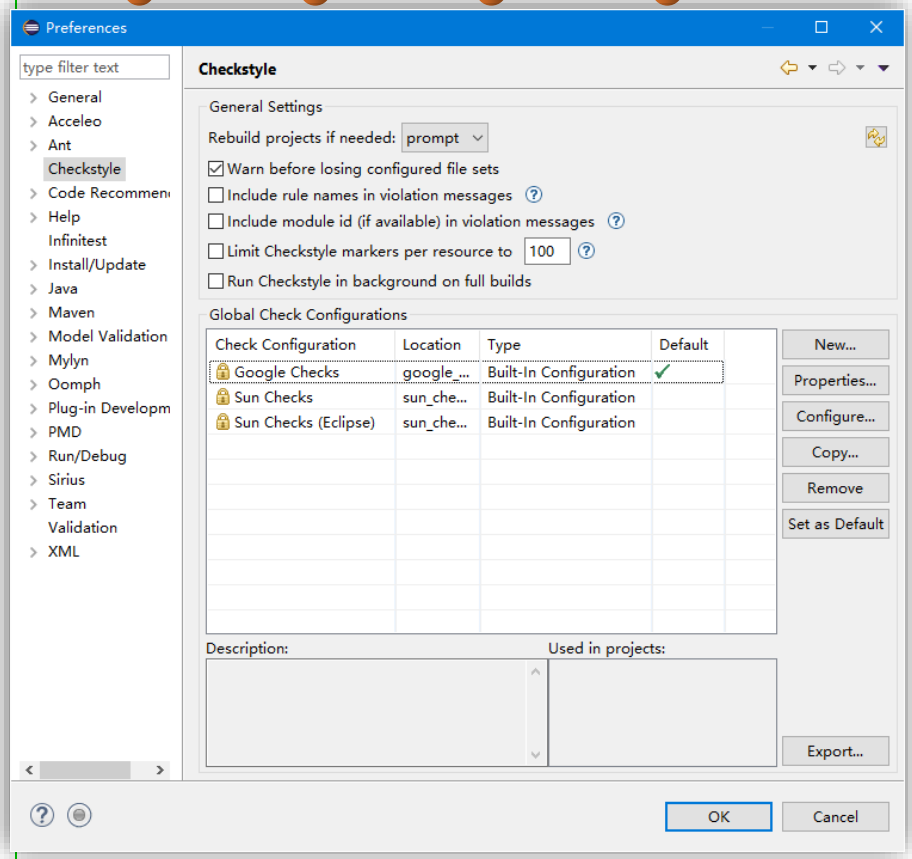
Problems | Javadoc | Declaration | Error Log

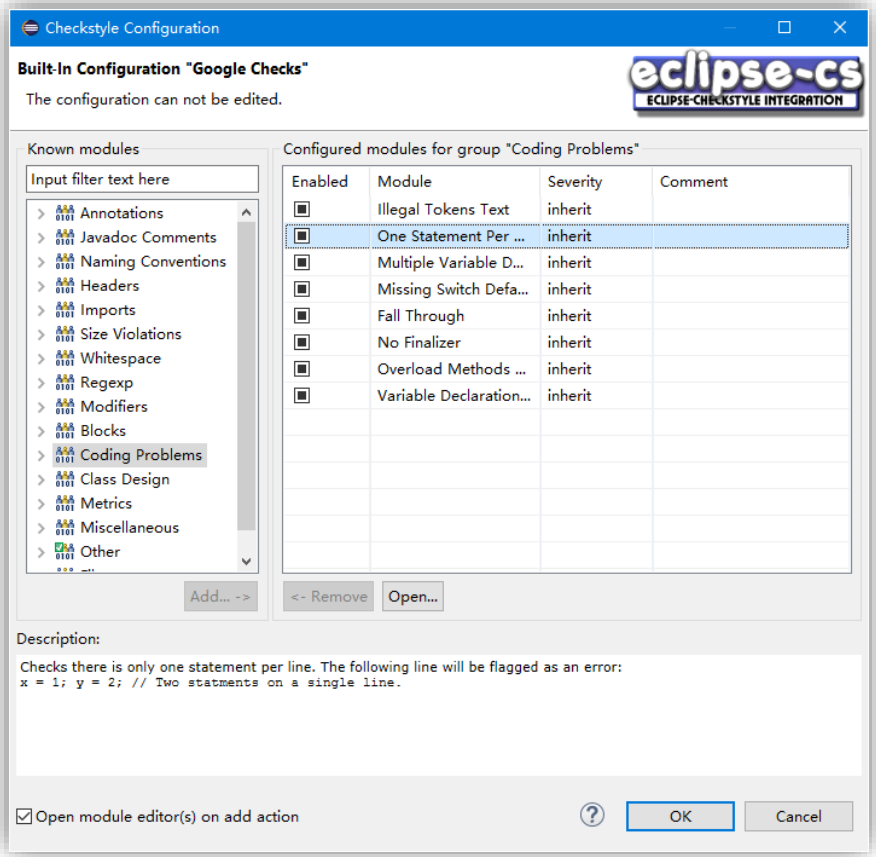
11 errors, 89 warnings, 0 infos (Filter matched 100 of 8,682 items)

| Description | Resource | In Folder | Location |
|---|---|---|---|
| '{' should be on the previous line. | Checker.j... | checkstyle/src/checkstyle/... | line 52 |
| First sentence should end with a peri... | Checker.j... | checkstyle/src/checkstyle/... | line 53 |
| First sentence should end with a peri... | Checker.j... | checkstyle/src/checkstyle/... | line 57 |
| First sentence should end with a peri... | Checker.j... | checkstyle/src/checkstyle/... | line 60 |
| First sentence should end with a peri... | Checker.j... | checkstyle/src/checkstyle/... | line 67 |
| First sentence should end with a peri... | Checker.j... | checkstyle/src/checkstyle/... | line 70 |
| First sentence should end with a peri... | Checker.j... | checkstyle/src/checkstyle/... | line 72 |
| First sentence should end with a peri... | Checker.j... | checkstyle/src/checkstyle/... | line 75 |

Writable | Smart Insert | 56 : 1

# Checkstyle in Eclipse



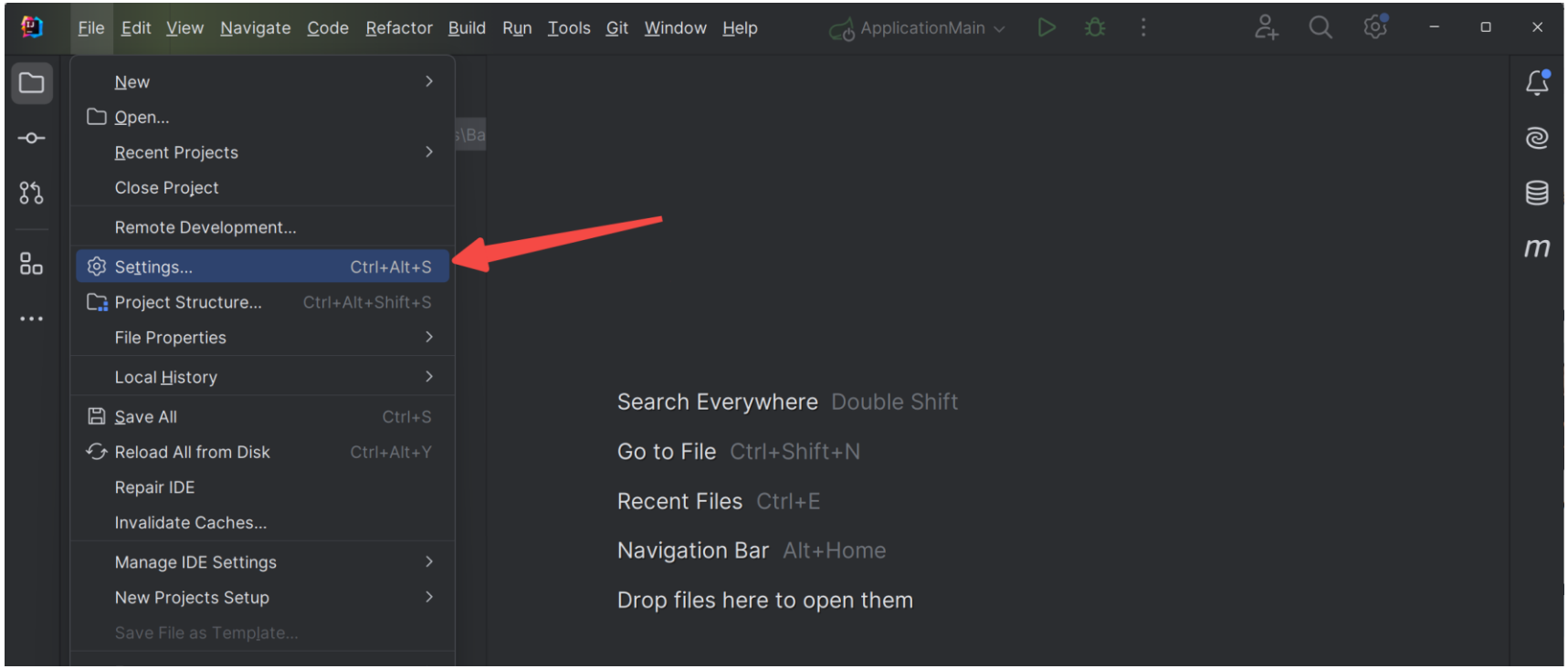




此处选择编码规范

此处阅读规范详情

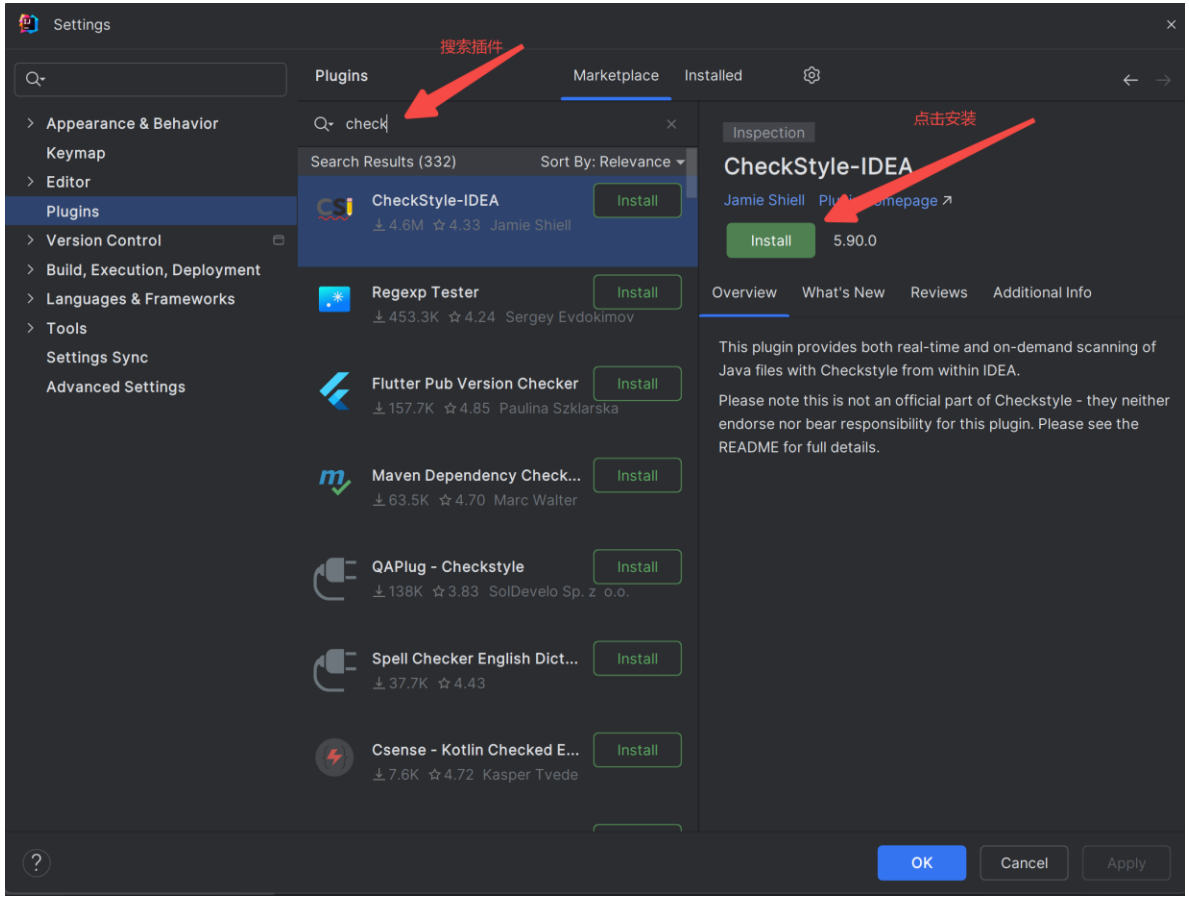
# Checkstyle in IDEA

- **IDEA下的安装**
  - **打开IDEA菜单栏Settings中的plugins，搜索CheckStyle-IDEA并安装**



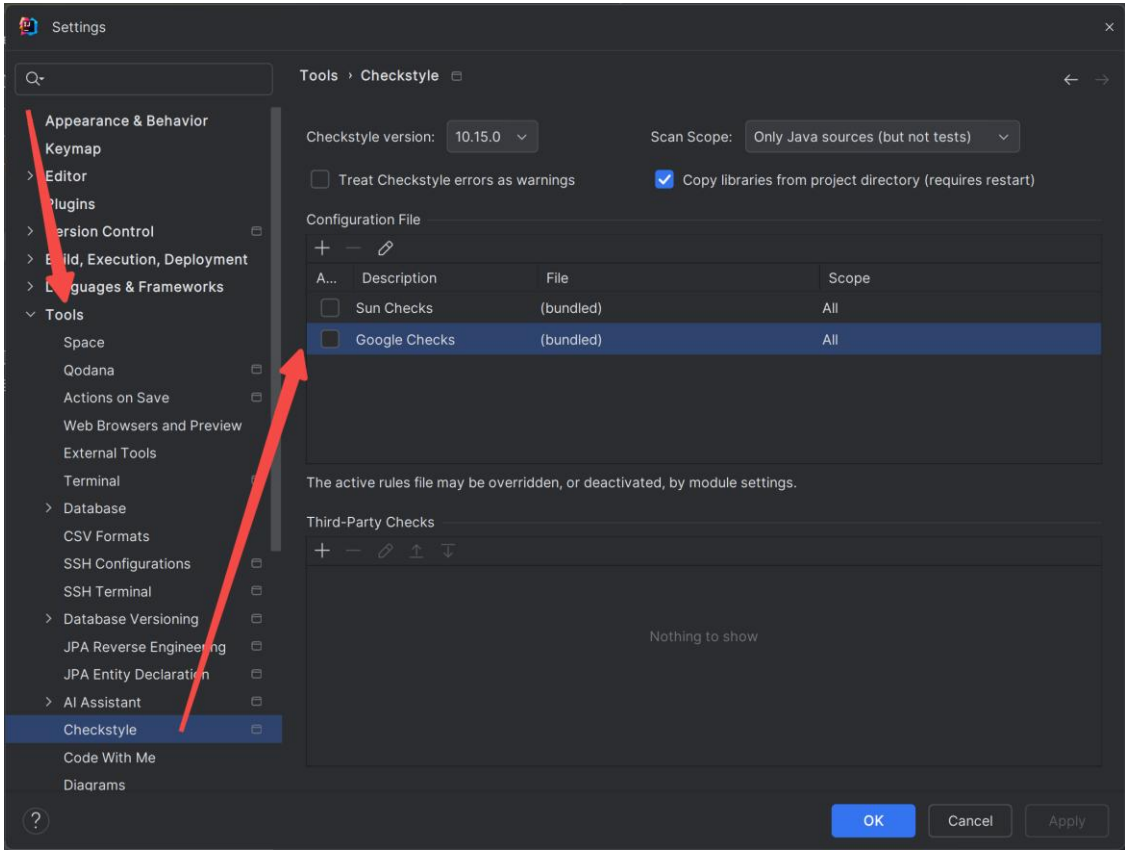
# Checkstyle in IDEA

- **IDEA下的安装**
  - **打开IDEA菜单栏Settings中的plugins，搜索CheckStyle-IDEA并安装**



# Checkstyle in IDEA

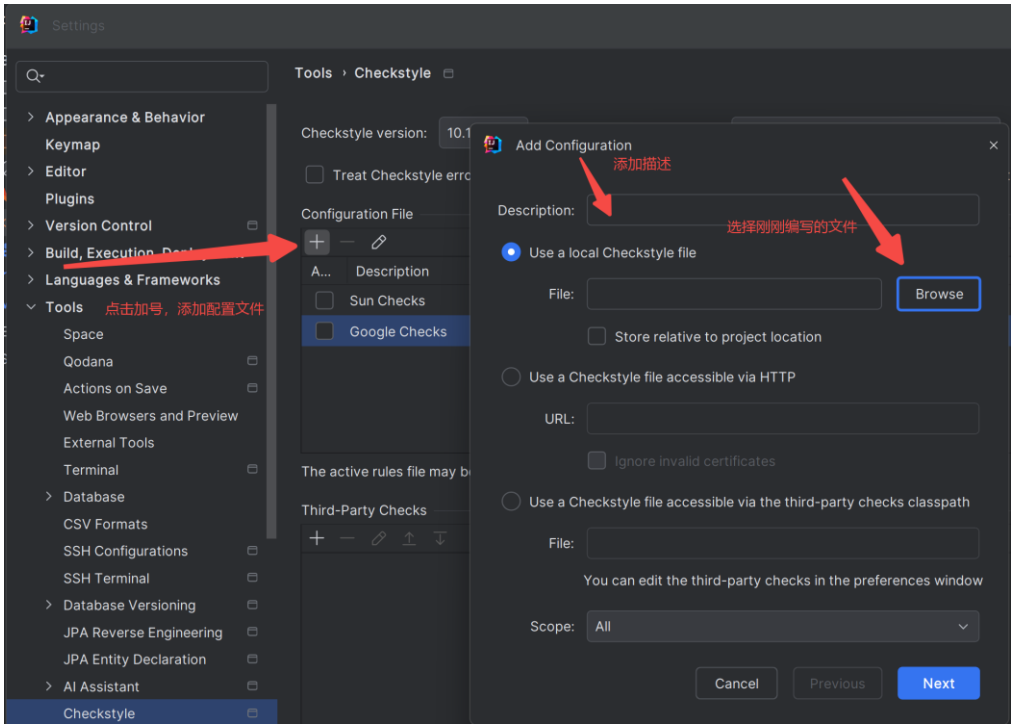
- **配置CheckStyle**
  - **按如下步骤点击，可以选择配置内置的配置文件，以Google Checks为例，选中后Apply**



# Checkstyle in IDEA

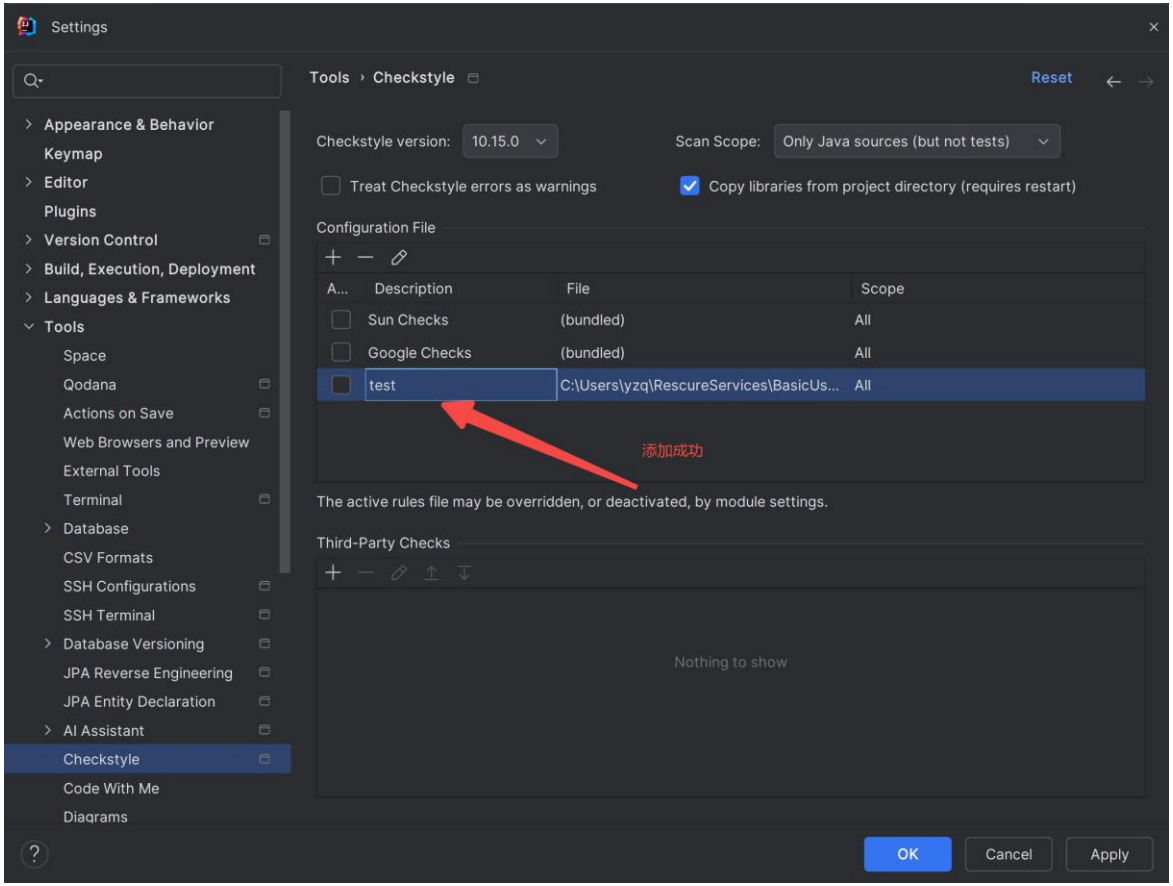
- **配置CheckStyle**
  - 也可以自己编写配置文件并导入，在本地新建checkstyle.xml文件，里面的内容示例如下：

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE module PUBLIC "-//Puppy Crawl//DTD Check Configuration 1.3/
/EN"
        "http://www.puppycrawl.com/dtds/configuration_1_3.dtd">
<module name="Checker">
    <!-- 官方主页: https://checkstyle.sourceforge.io -->
    <!-- 检查文件是否以一个空行结束 -->
    <module name="NewlineAtEndOfFile"/>
    <!-- 文件长度不超过1500 行 -->
    <module name="FileLength">
        <property name="max" value="1500"/>
    </module>
    <!-- 每行不超过120 个字 -->
    <module name="LineLength">
        <property name="max" value="150"/>
    </module>
    <module name="TreeWalker">
        <!-- import 中避免星号"*" -->
        <module name="AvoidStarImport"/>
        <!-- 检查是否从非法的包中导入了类 -->
        <module name="IllegalImport"/>
        <!-- 检查是否导入了多余的包 -->
        <module name="RedundantImport"/>
    </module>
</module>
```



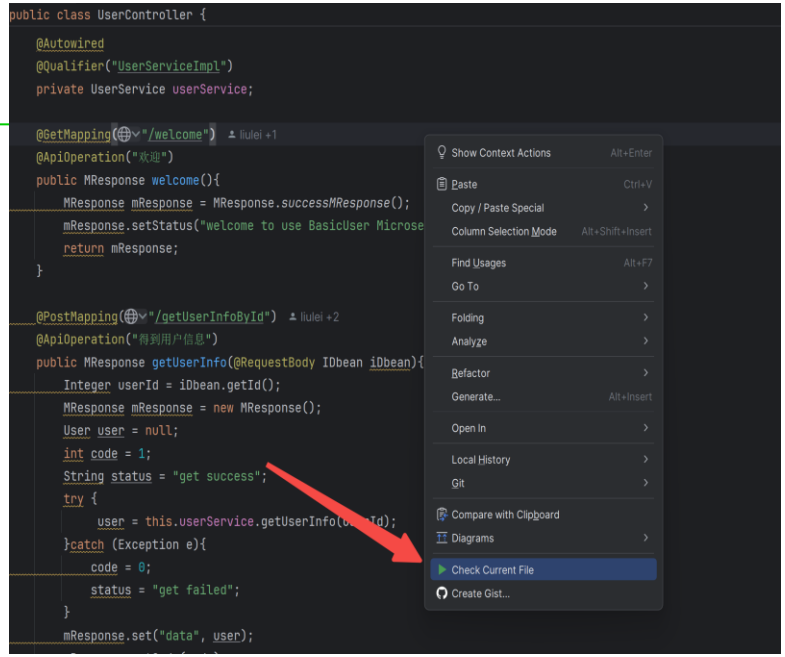
# Checkstyle in IDEA

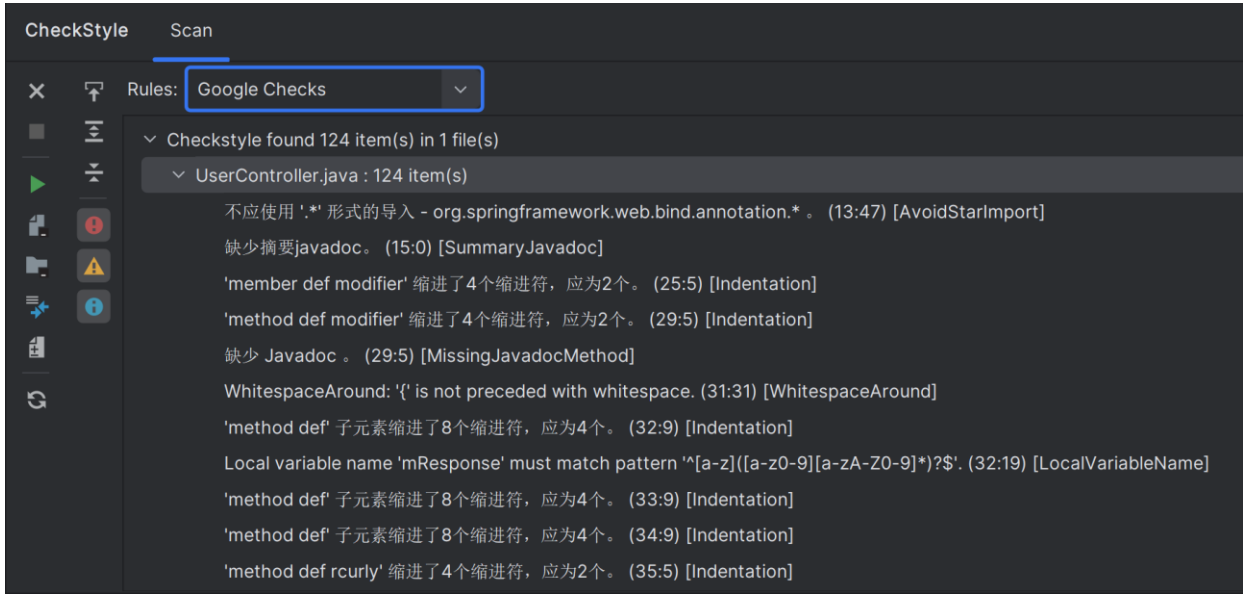
- **配置CheckStyle**
  - 勾选指定的规范使之生效。



# Checkstyle in IDEA

- 使用
  - 打开某一代码文件，右键点击Check Current File勾选指定的规范使之生效。



# CheckStyle的帮助

- 从https://checkstyle.org/index.html学习：
  - 如何安装和配置
  - 如何自定义配置
  - 每个问题的具体含义以及如何修正
- **由此理解Java编码规范。**

checkstyle

| Last Published: 2024-05-26 | Version: 10.17.0

**About**
- **Checkstyle**
- Release Notes
- Consulting
- Sponsoring

**Documentation**
- Configuration
  - Property Types
  - System Properties
- Running
  - Ant Task
  - Command Line
- Checks
  - Annotations
  - Block Checks
  - Class Design
  - Coding
  - Headers
  - Imports
  - Javadoc Comments
  - Metrics
  - Miscellaneous
  - Modifiers
  - Naming Conventions
  - Regexp
  - Size Violations
  - Whitespace
- Filters
- File Filters
- Style Configurations
  - Google's Style

## Overview

Checkstyle is a development tool to help programmers write Java code that adheres to a coding standard. It automates the process of checking Java code to spare humans of this boring (but important) task. This makes it ideal for projects that want to enforce a coding standard.

Checkstyle is highly configurable and can be made to support almost any coding standard. An example configuration files are supplied supporting the Sun Code Conventions, Google Java Style.

A good example of a report that can be produced using Checkstyle and Maven can be seen here .

## Features

Checkstyle can check many aspects of your source code. It can find class design problems, method design problems. It also has the ability to check code layout and formatting issues.

For a detailed list of available checks please refer to the Checks page.

## Download

The latest release of Checkstyle can be downloaded from the GitHub releases page, or Maven central.

If you want to live on the bleeding edge, you can checkout the current development code from GitHub and compile yourself. Or use jitpack.io to build and get artifacts you need (you can even build your forked repo).

## Previous Version Documentation

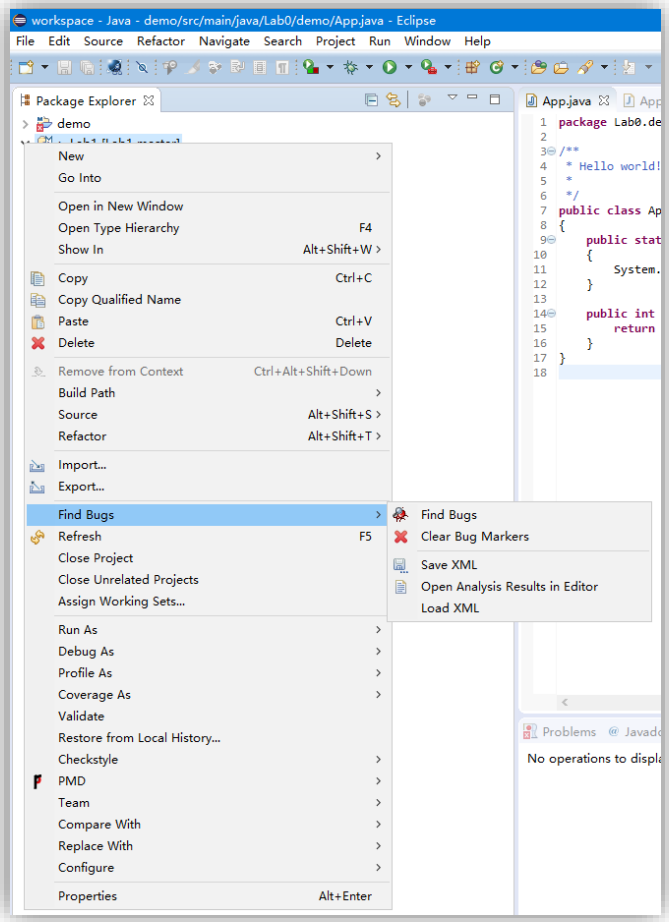
Current site contains documentation only for latest, released version. We only support this latest version. We highly recommend read latest documentation first. However, you might need documentation for previous releases because you are still using an older version and latest version describe some features that does not work for you.
You can find documentation for most old versions using a URL format like https://checkstyle.sourceforge.io/version/X.X where "X.X" is the version number.

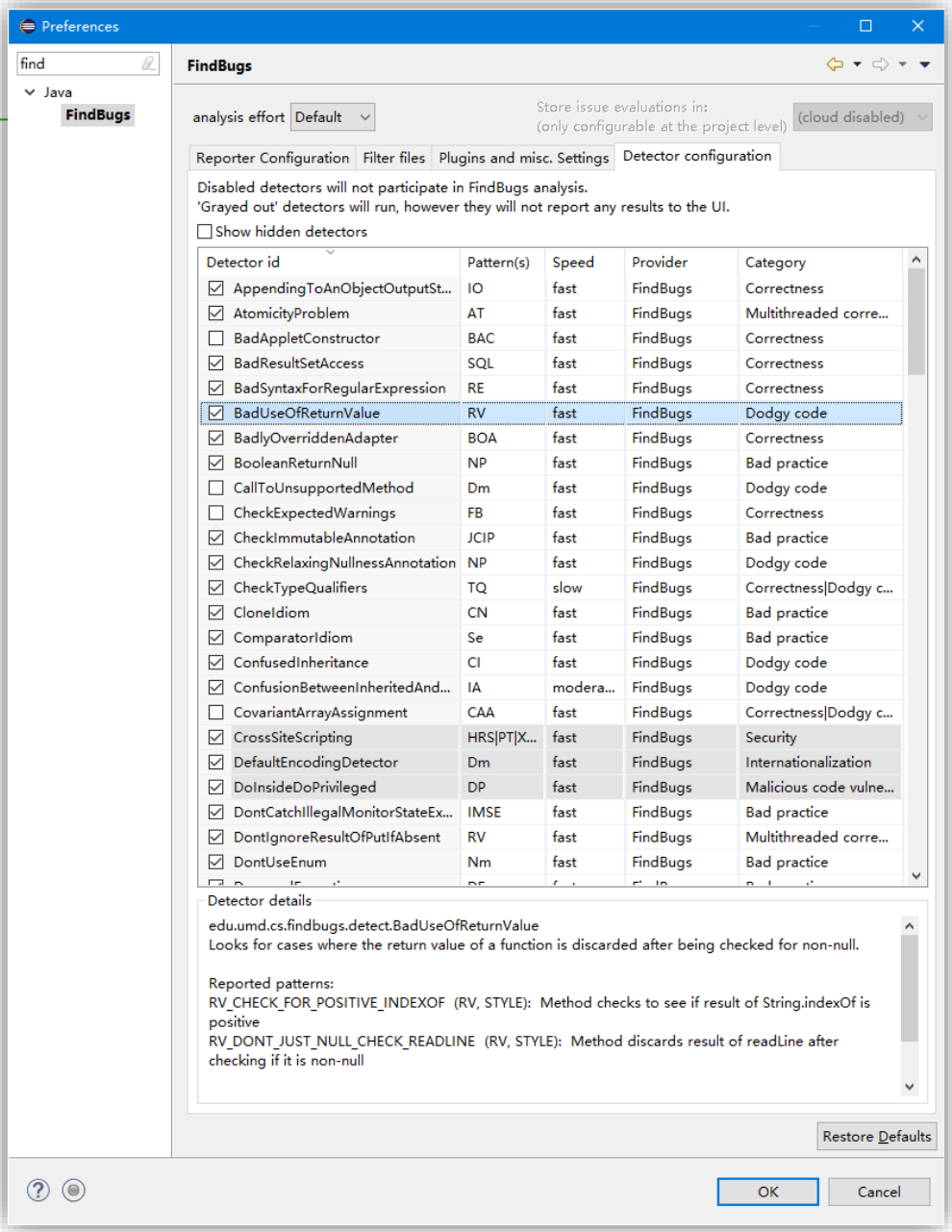
# SpotBugs

- **SpotBugs：Java静态代码分析工具，不注重样式或者格式，专注于寻找真正的缺陷或者潜在的性能问题，帮助开发者提高代码质量以及排除隐含的缺陷，可以在不实际运行程序的情况对软件进行分析。** https://spotbugs.github.io
- **常见的检测类型如下：**
  - 正确性(Correctness)：在某种情况下会导致bug，如错误的强制类型转换等。
  - 最佳实践反例(Bad practice)：这种类别下的代码违反了公认的最佳实践标准，比如某个类实现了equals方法但未实现hashCode方法等。
  - 多线程正确性(Multithreaded correctness)：关注于同步和多线程问题。
  - 性能(Performance)：潜在的性能问题。
  - 安全(Security)：安全相关。
  - 高危(Dodgy)：该类型下的问题代码导致bug的可能性很高。

# SpotBugs in Eclipse

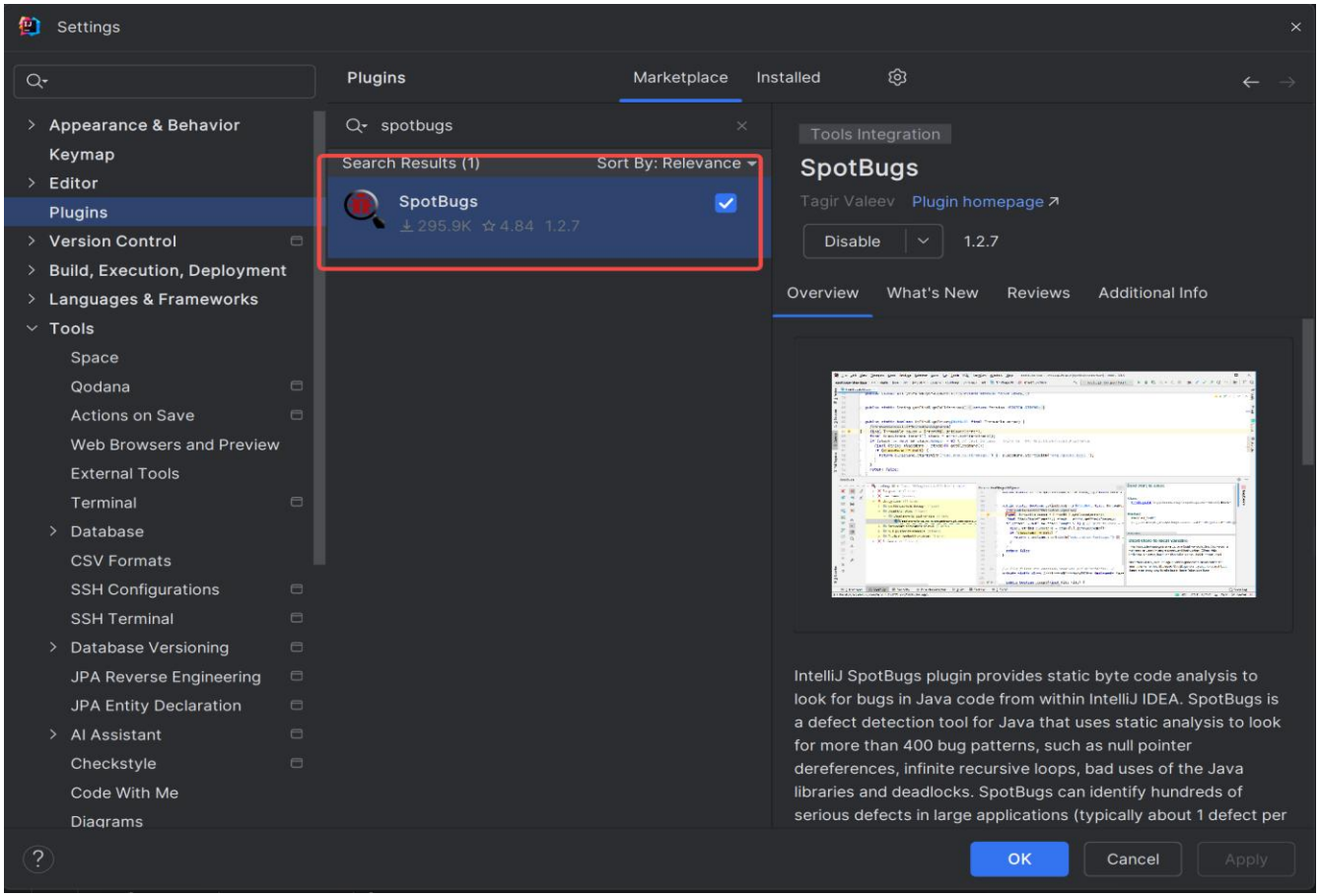
- 通过Eclipse Marketplace安装使用。



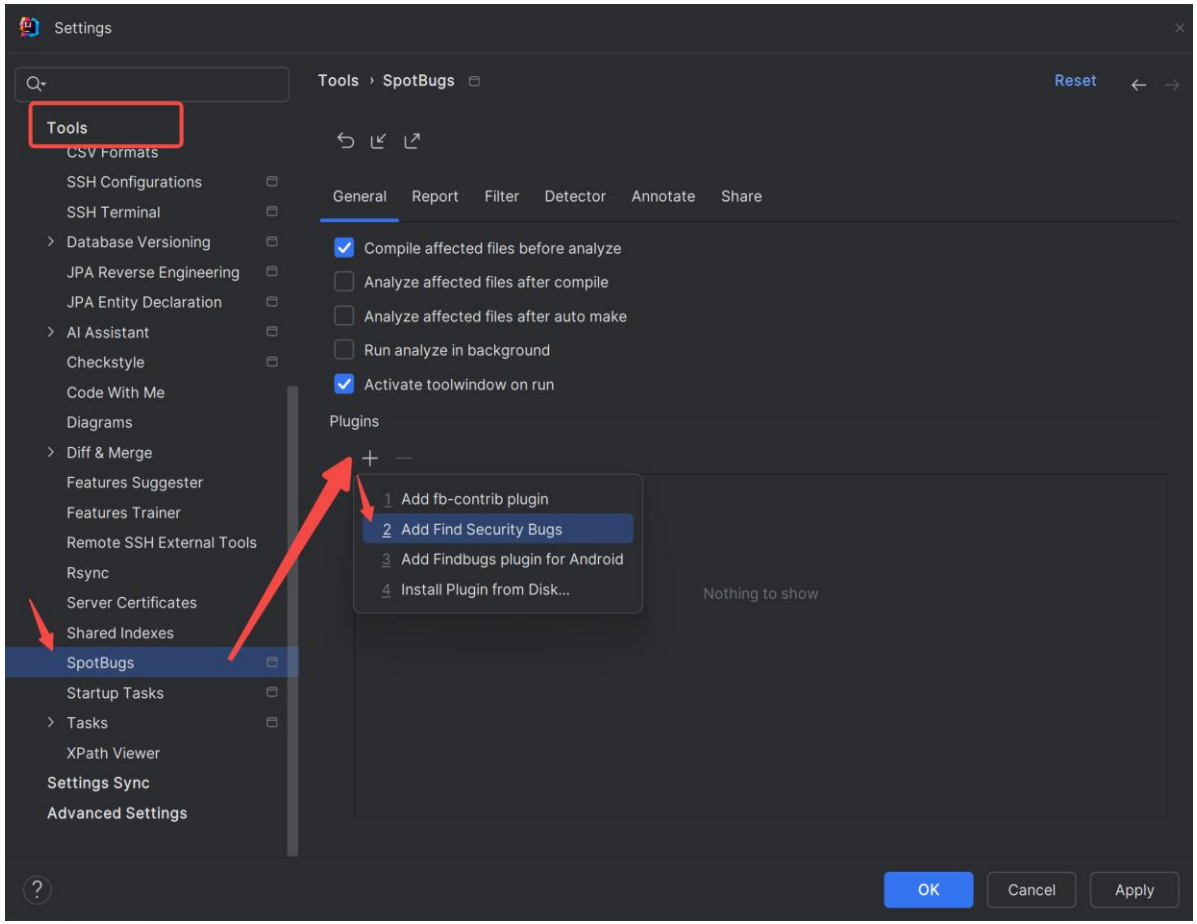
# SpotBugs in IDEA

- 安装
  - 在IDEA插件市场搜索SpotBugs，点击Install安装，安装后重启IDEA。



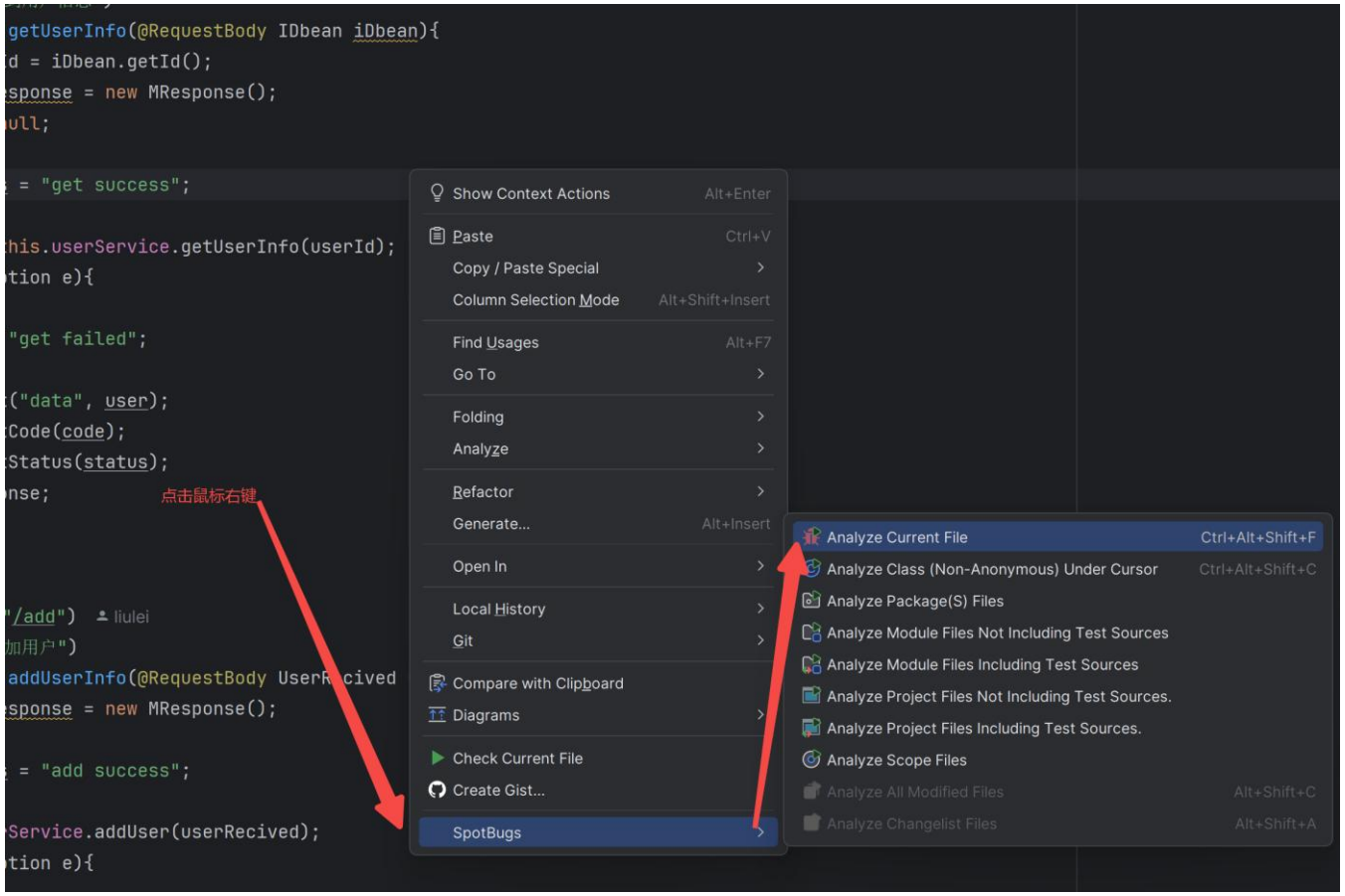
# SpotBugs in IDEA

- **配置安全规则**
  - 在设置的Tools下，点击SpotBugs，按下图所示点击加号，添加规则。



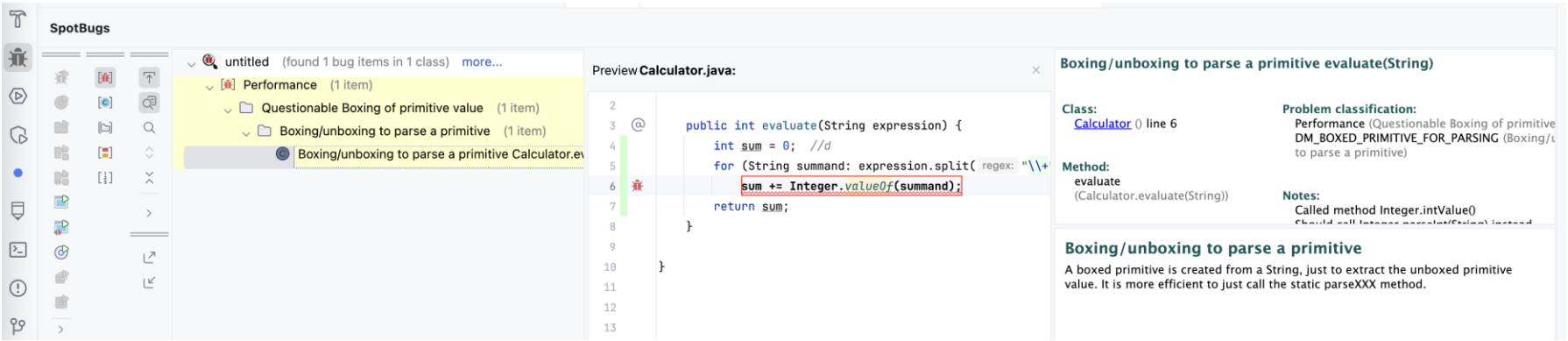
# SpotBugs in IDEA

- 使用
  - 在某一代码文件上，右键选择Analyze Current File。



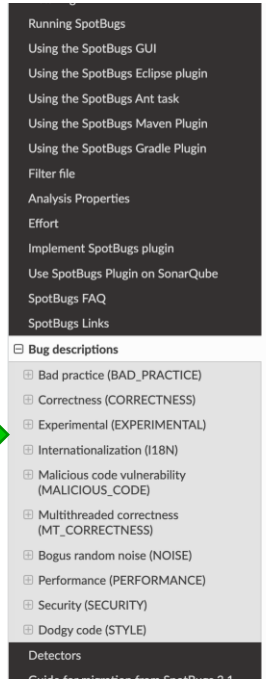
# SpotBugs in IDEA

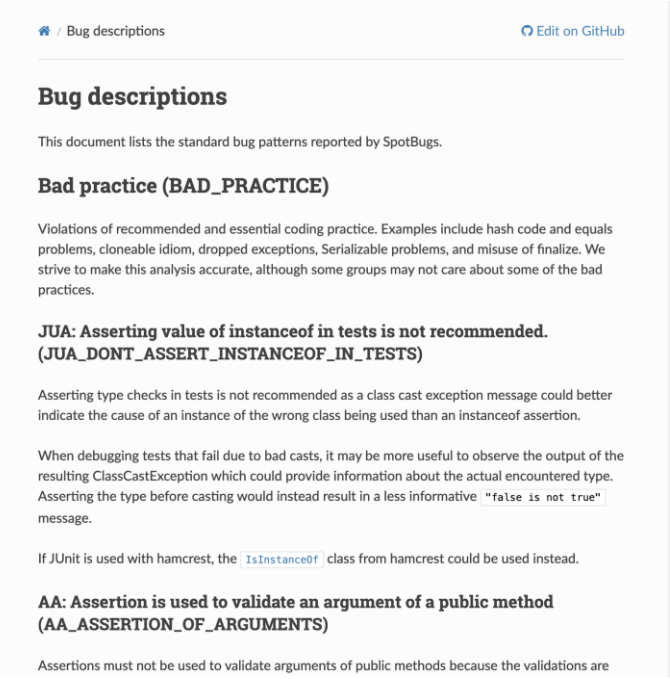
- 使用
  - 在某一代码文件上，右键选择Analyze Current File。
  - 若代码中包含潜在BUG，会在控制台输出



# SpotBugs in IDEA

- 阅读网站 https://spotbugs.github.io 了解详细信息
- 阅读Bug description学习常见Bug和相关知识



# 代码评审部分：评判标准

- 是否可在IDE中顺利配置工具并启动执行；
- 能够充分理解CheckStyles、SpotBugs工具所检测的不符合规范的常见代码问题；
- 根据工具报告的问题，对代码做了充分的改进；
- 实验报告的规范性、完整性。

哈爾濱工業大學
HARBIN INSTITUTE OF TECHNOLOGY

# 单元测试

# 实验目标

- 针对Lab1完成的程序，设计黑盒测试用例和白盒测试用例。

- 在JUnit环境下撰写测试代码并执行测试。
- 使用EclEmma或IDE自带工具统计测试的覆盖度。

- 注：如果使用Python等其他语言，自行选取对应的工具完成实验，例如Unittest、Pytest等单元测试工具， pytest-cov、 coverage.py等测试覆盖度统计工具。

# Step 1：设计黑盒测试用例

- **利用黑盒测试的等价类和边界值方法，为Lab1待测程序设计一组测试用例。**
- **测试对象(五选一)：**
  - `String queryBridgeWords(String word1, String word2)`：查询桥接词
  - `String generateNewText(String inputText)`：根据bridge word生成新文本
  - `String calcShortestPath(String word1, String word2)`：计算两个单词之间的最短路径
  - `Double calPageRank(String word)`：计算单词的PR值
  - `String randomWalk()`：随机游走
- **根据Lab1的要求，给出所选被测函数的需求规约描述；**
- **每个测试用例由输入数据和期望输出两部分组成。**

# Step 2：使用JUnit编写黑盒测试用例并执行

- **在Lab1的Git仓库里，建立新的Git分支，命名为Lab3b (b代表black-box testing)；**
- **针对每个测试用例撰写testcase；**
- **执行测试用例：**
  - 执行，产生结果，记录实际输出；
  - 记录、分析结果；
  - 针对失败的测试用例，发现代码的问题，并修改代码；
- **重复上一步，直到所有的测试用例都完全通过为止。**
- **将Lab3b合并到master分支，并推送至GitHub。**

# Step 3：设计白盒测试用例

- **针对Lab3b分支的当前代码，对以下函数(三选一)使用基本路径法设计白盒测试用例：**
  - `String queryBridgeWords(type G, String word1, String word2)` 查询桥接词
  - `String calcShortestPath(type G, String word1, String word2)`：计算两个单词之间的最短路径
  - `String randomWalk(type G)`：随机游走

- **每个测试用例由输入数据和期望输出两部分组成。**

# Step 4：使用JUnit编写并执行白盒测试代码

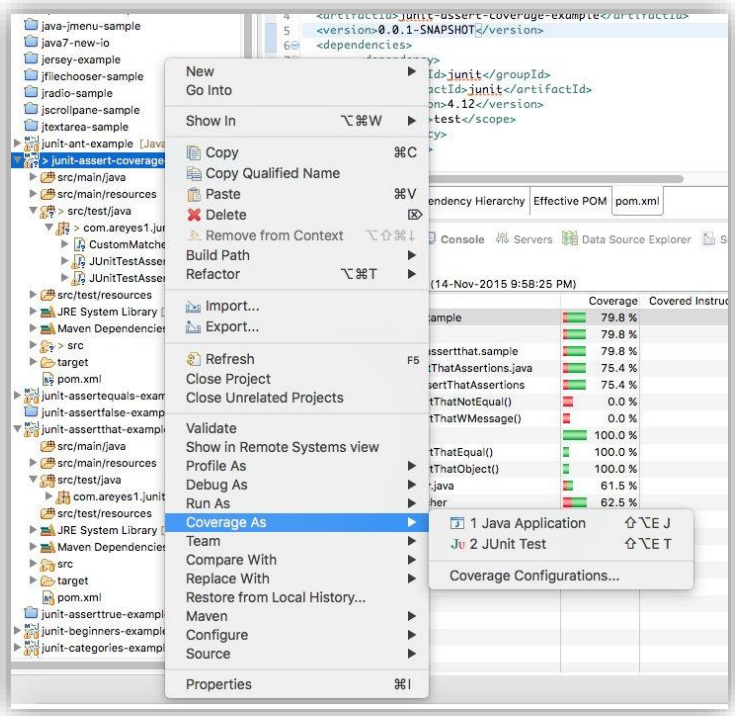
- **在Lab1的Git仓库里，建立新的Git分支，命名为Lab3w (w代表white-box testing)；**
- **针对每个测试用例撰写testcase；**
- **执行测试用例：**
  - 执行，产生结果，记录实际输出；
  - 记录、分析结果；
  - 针对失败的测试用例，发现代码的问题，并修改代码；
- **重复上一步，直到所有的测试用例都完全通过为止。**
- **将Lab3w合并到master分支，并推送至GitHub。**

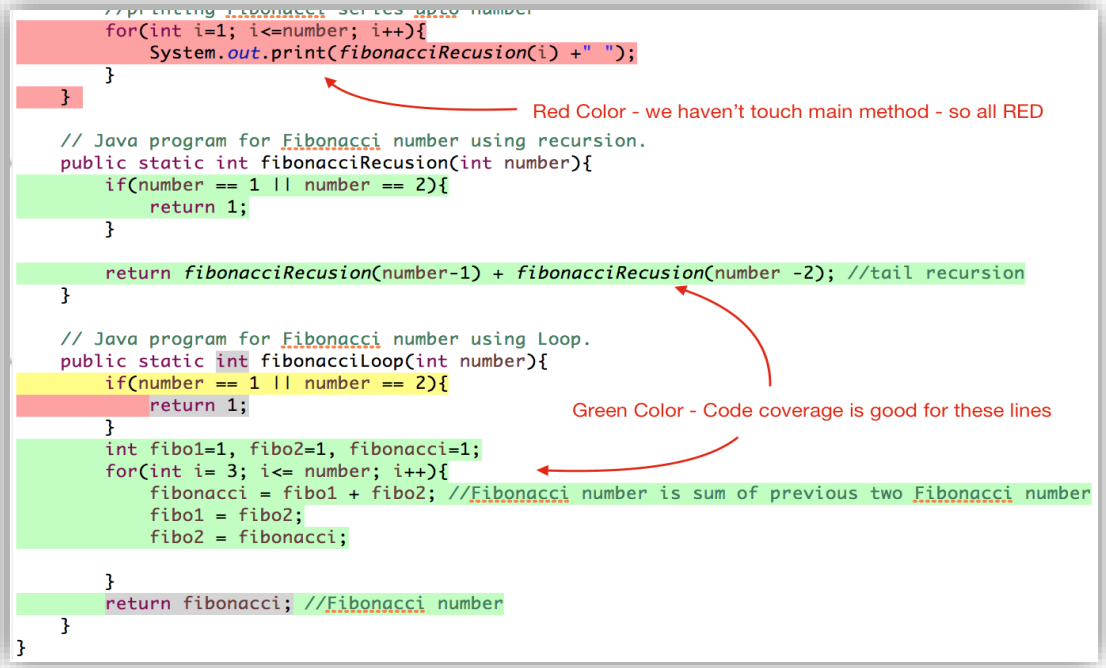
# Step 5：统计测试覆盖度

- 测试的覆盖率是测试质量的一个重要指标。
  - 可使用EclEmma工具 https://www.eclemma.org/
  - 或Eclipse自带Coverage分析工具
  - 或IDEA自带Coverage分析工具
- 在测试过程中，当运行测试程序，覆盖度分析工具可自动分析出被测程序的各行代码被覆盖的情况；
- 代码被覆盖得越全面，测试质量就越好。
- 从工具导出覆盖度分析报告，观察语句覆盖度。

# Step 5：统计测试覆盖度

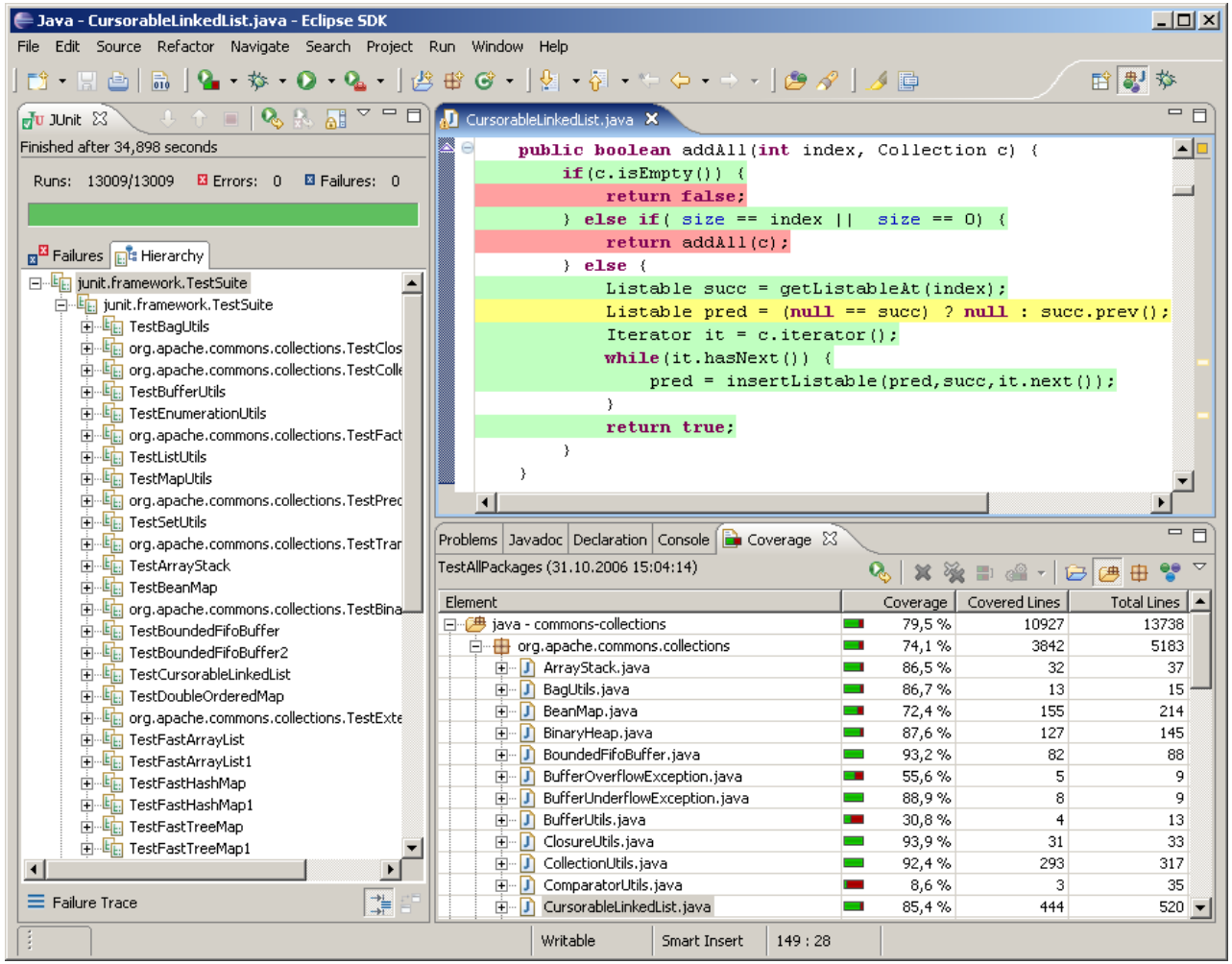
- **Eclispe下通过Marketplace安装EclEmma**



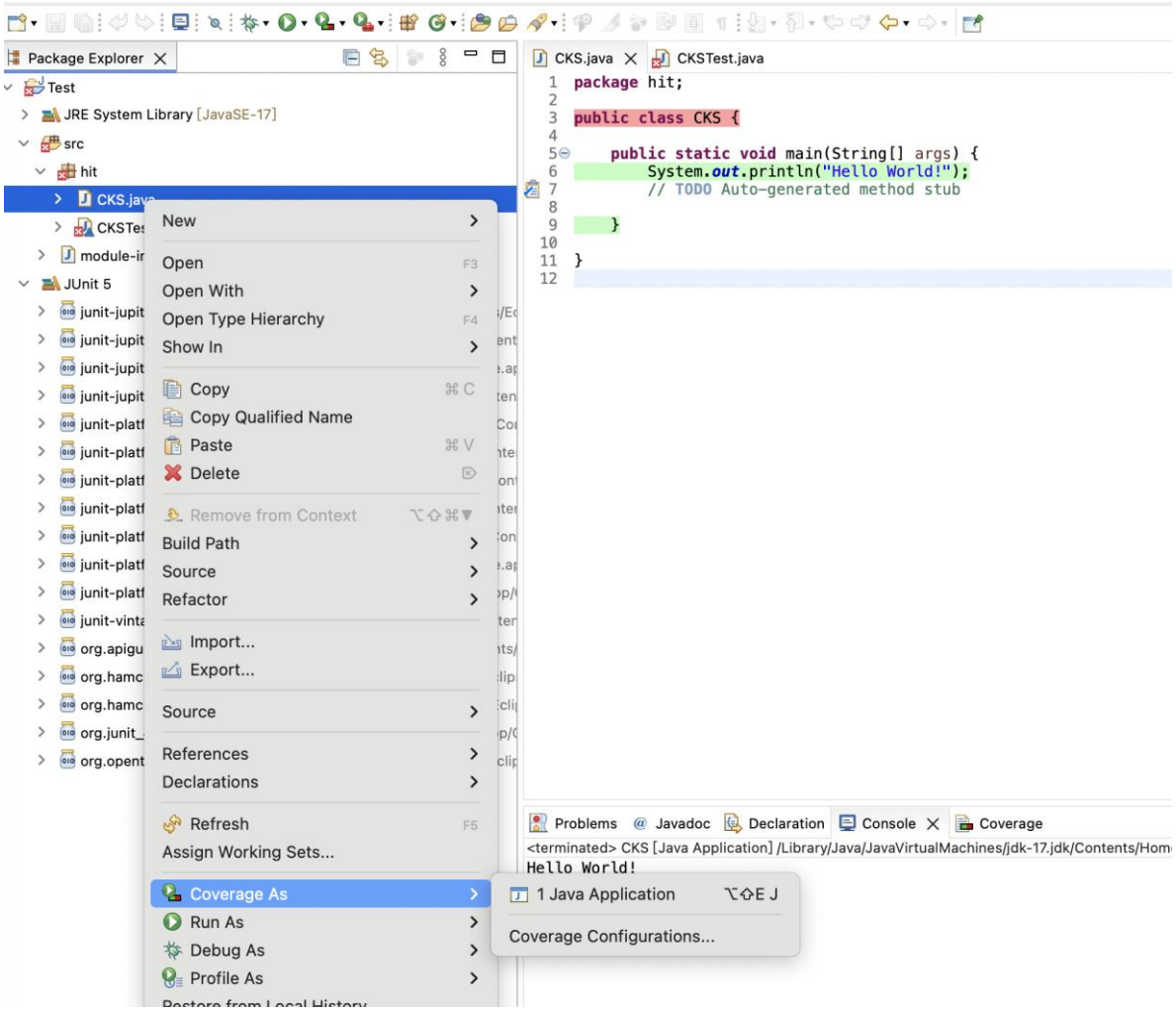
# Step 5：统计测试覆盖度

- **Eclispe下通过Marketplace安装EclEmma**



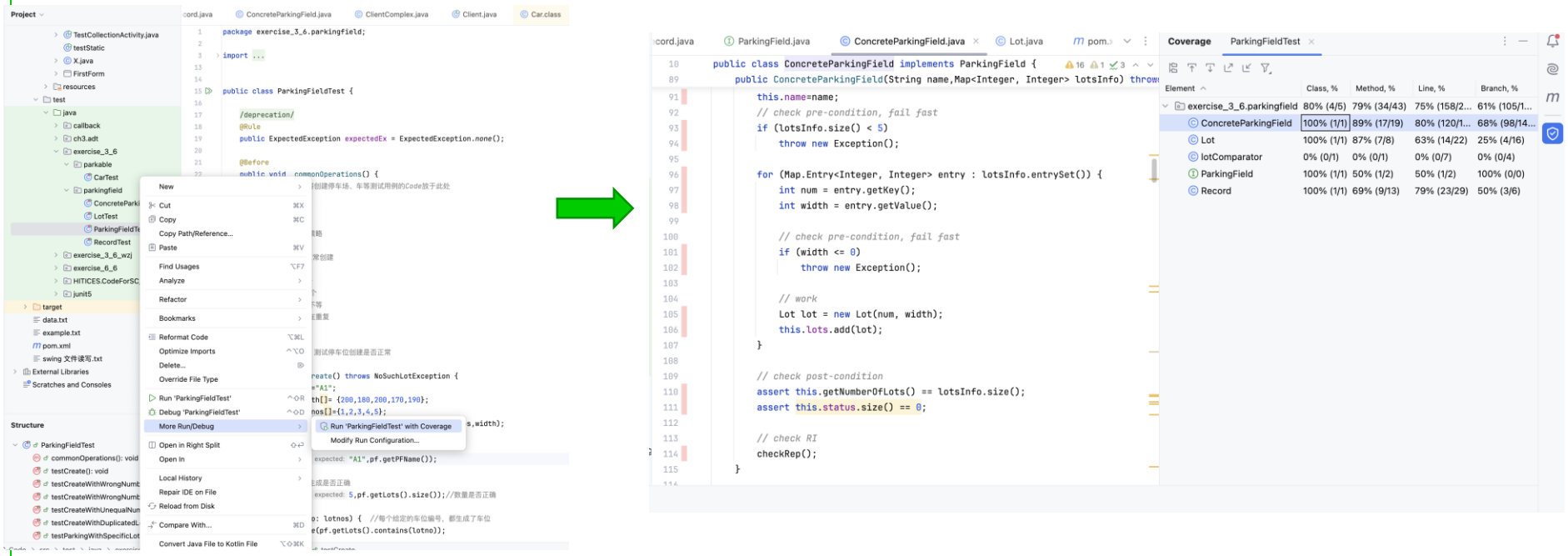
# Step 5：统计测试覆盖度

- 采用Eclispe下自带的覆盖度分析工具



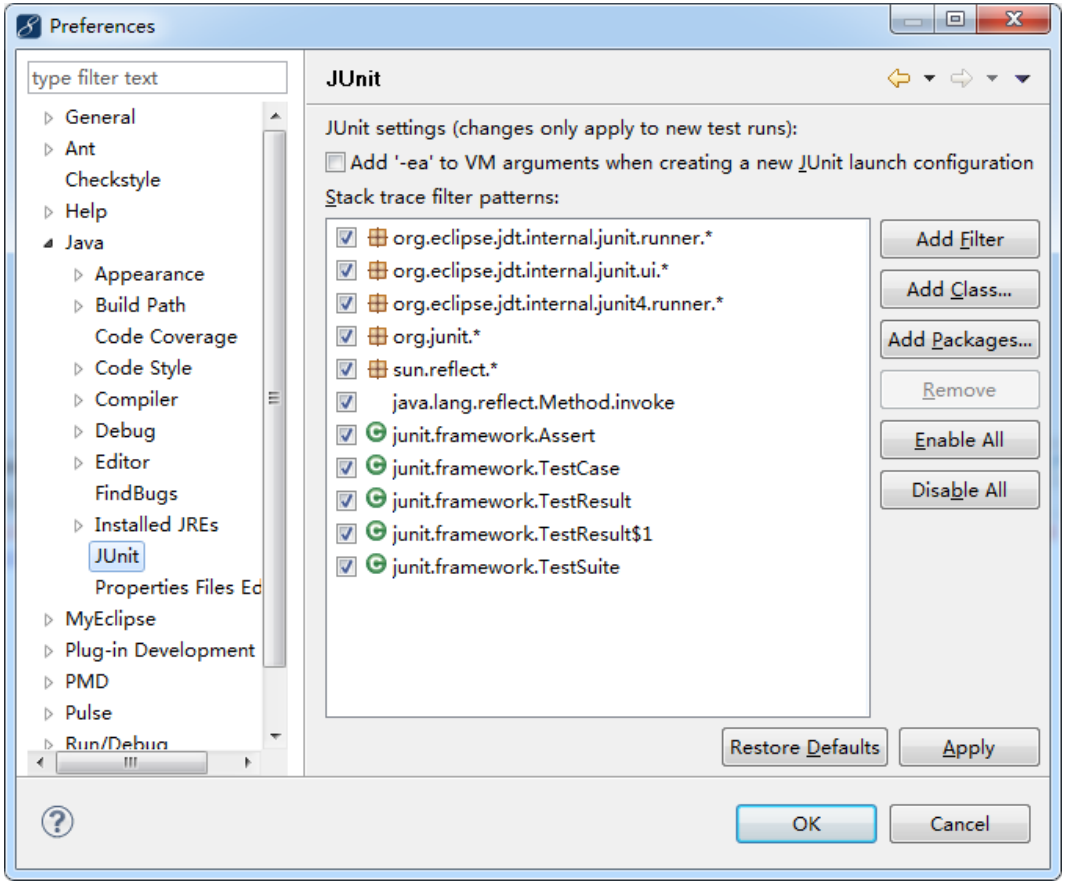
# Step 5：统计测试覆盖度

- **IDEA下自带覆盖度分析工具**
  - https://www.jetbrains.com/help/idea/code-coverage.html



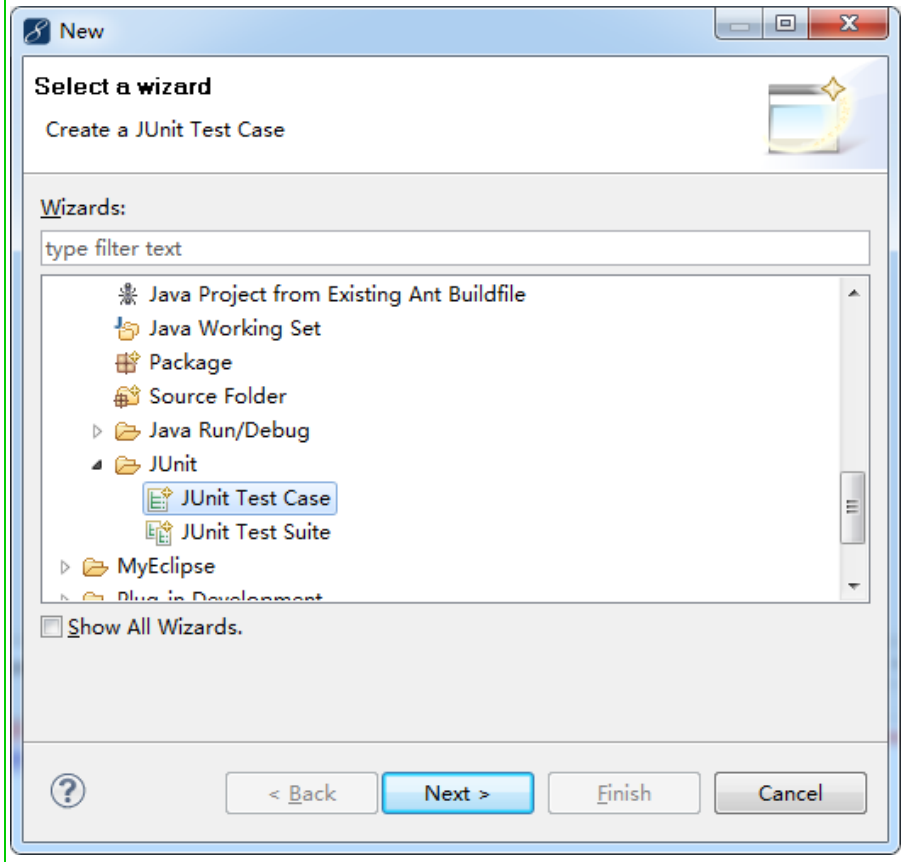
# JUnit的安装与配置

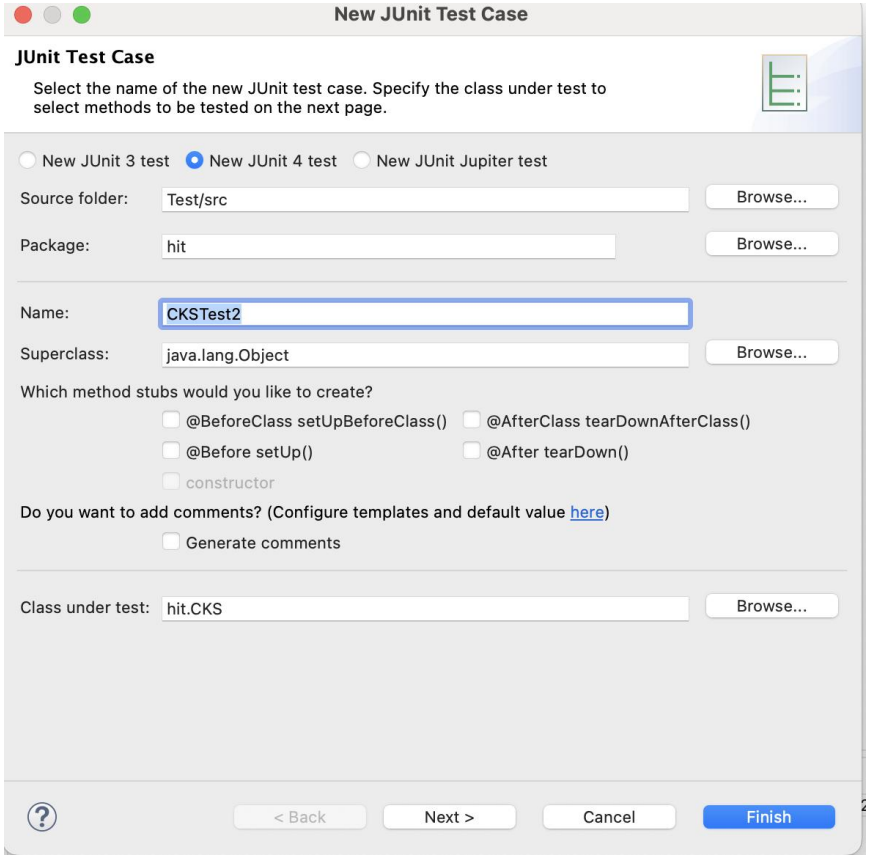
- https://junit.org/junit4/ https://junit.org/junit5/
- **Eclipse和IDEA当前版本已经集成了Junit。**



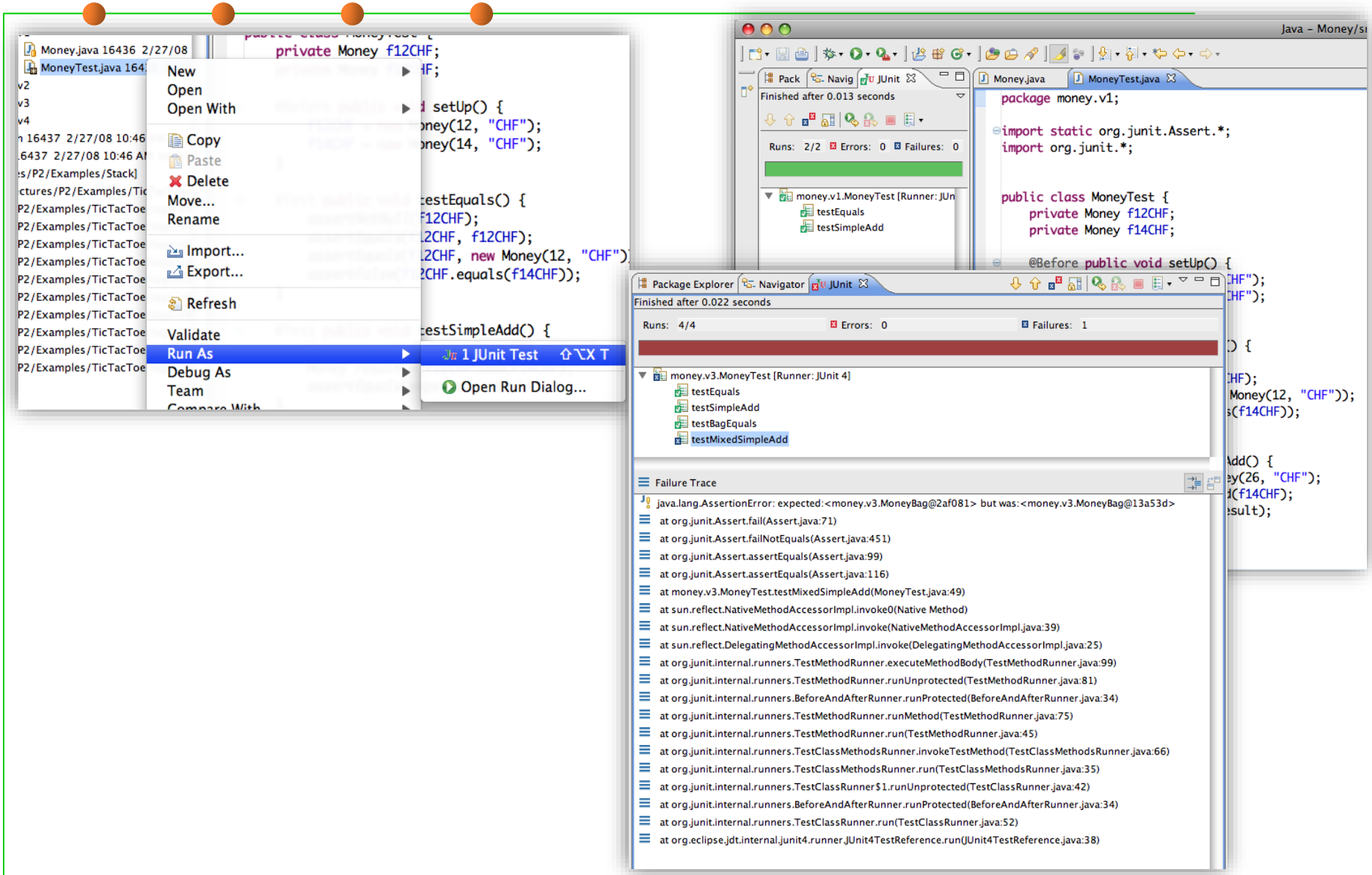
# JUnit的安装与配置 Eclipse

- 针对一个存在的项目，在其中新建JUnit测试用例或测试套件；



# 在Eclipse中运行JUnit测试用例



# JUnit的安装与配置 IDEA

- **IDEA下使用JUnit https://junit.org/junit4/**
  - 方法1：下载 junit.jar 和 hamcrest-core.jar ，加入项目的Libraries中
  - 方法2：如果使用Maven，见网站说明

junit-team / junit4

Type / to search

Issues 107 | Pull requests 18 | Actions | Projects | Wiki | Security 1 | Insights

## Download and Install

Marc Philipp edited this page on Sep 29, 2023 · 35 revisions

To download and install JUnit you currently have the following options.

### Plain-old JAR

Download the following JARs and add them to your test classpath:

- `junit.jar`
- `hamcrest-core.jar`

### Maven

Add a dependency to `junit:junit` in `test` scope. (Note: 4.13 is the latest stable version as of the latest edit on this page.)

```
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.13</version>
  <scope>test</scope>
</dependency>
```

### Gradle

See Use-with-Gradle

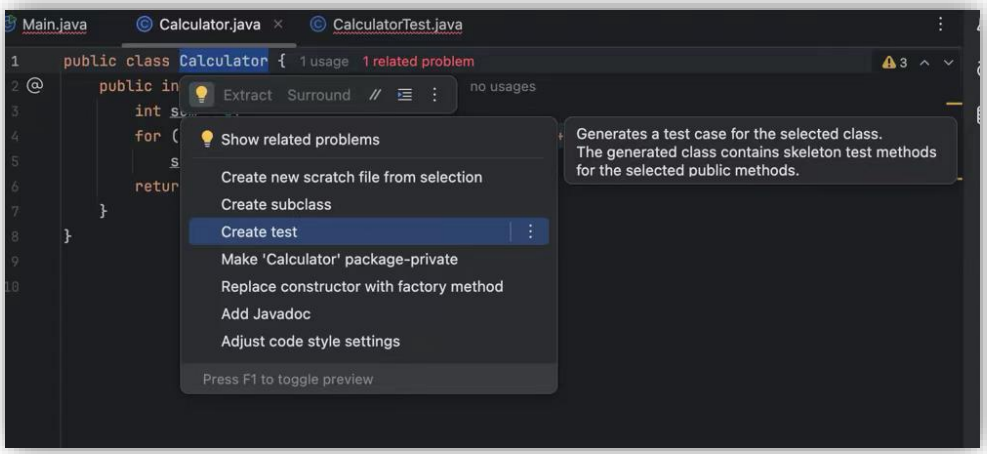
Pages 35

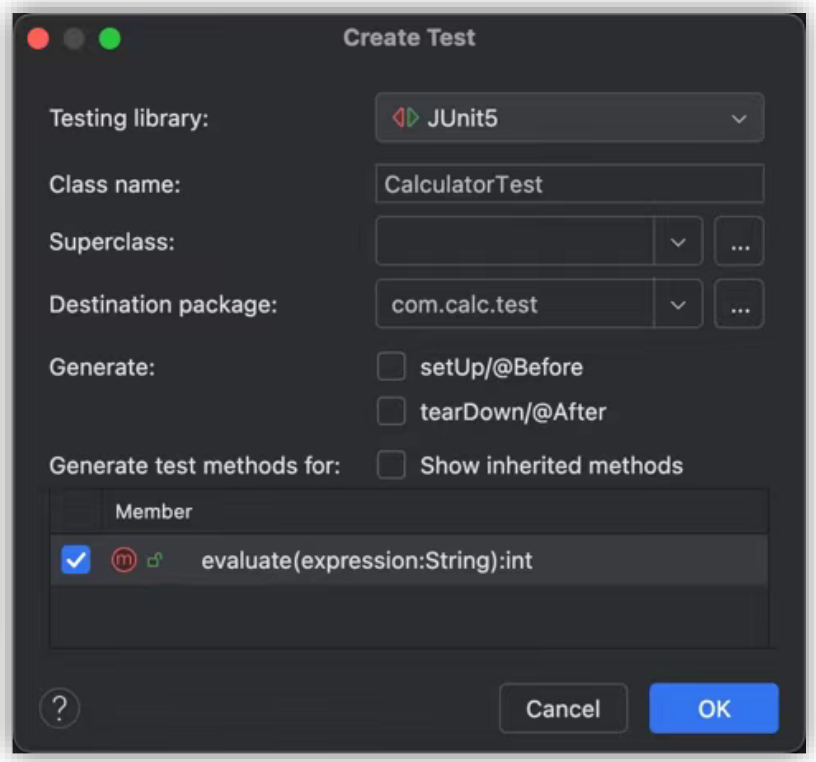
Find a page...

- Home
- 'Enclosed' test runner example
- Aggregating tests in suites
- Assertions
- Assertions CN
- Assumptions with assume
- Categories
- Continuous testing
- Custom runners
- Deprecation Policy
- Developing with fast tests
- Download and Install
  - Plain-old JAR
  - Maven
  - Gradle
- Exception testing

# JUnit的安装与配置 IDEA

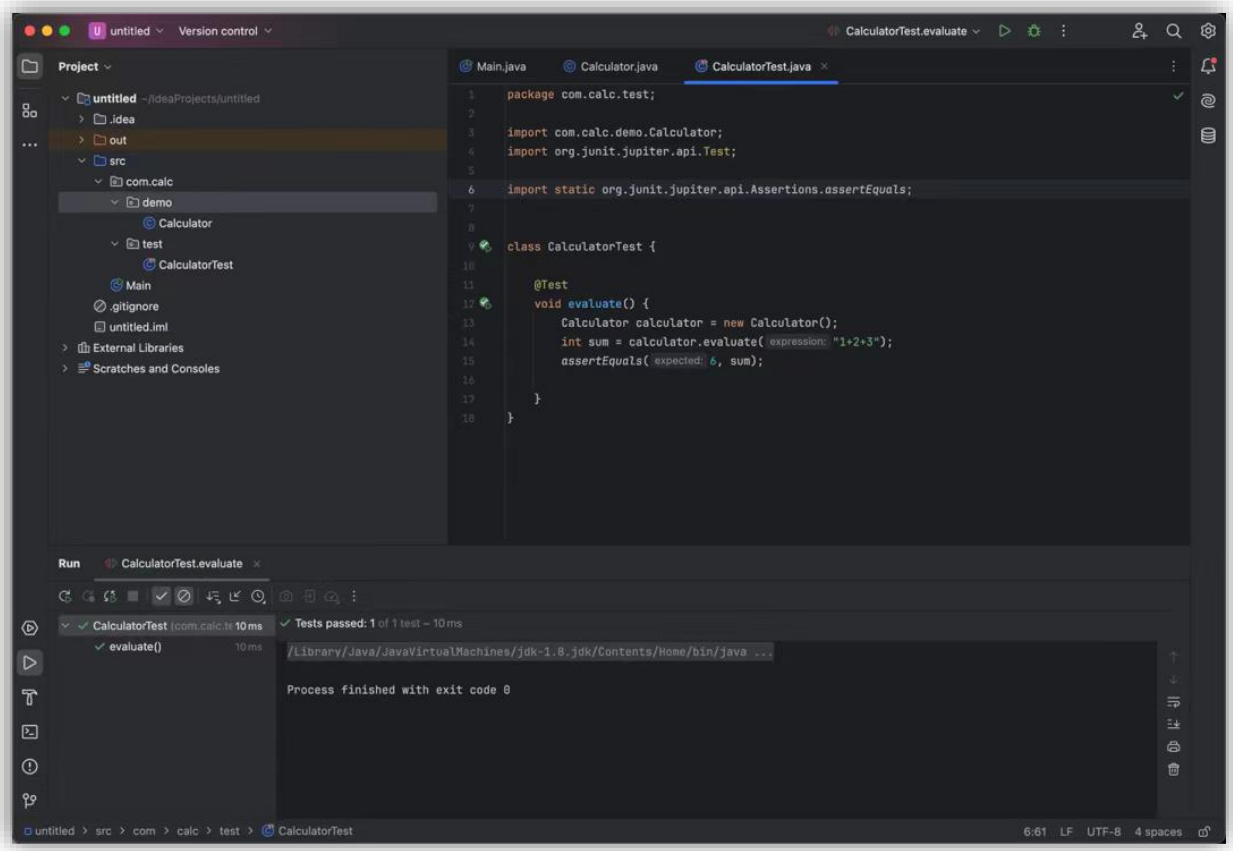
- **IDEA下使用JUnit https://junit.org/junit4/**
  - 方法3：在需要测试的类或接口名称上使用 Ctrl+Shift+t，然后选择创建测试。



# JUnit的安装与配置 IDEA

- **IDEA下运行JUnit**
  - 在测试方法点击 IDEA 运行图标（或用 Ctrl+Shift+F10）运行单元测试。



# JUnit4测试用例：小例子

```
public class Calculator {
  public int evaluate(String expression) {
    int sum = 0;
    for (String summand: expression.split("\\+"))
      sum += Integer.valueOf(summand);
    return sum;
  }
}

------

import static org.junit.Assert.assertEquals;
import org.junit.Test;

public class CalculatorTest {
  @Test
  public void evaluatesExpression() {
    Calculator calculator = new Calculator();
    int sum = calculator.evaluate("1+2+3");
    assertEquals(6, sum);
  }
}
```

# 单元测试部分：实验评判标准

- 是否可正确配置JUnit/EclEmma；
- 所设计的测试用例是否满足要求；
- 是否正确书写了JUnit测试代码并执行，以获得测试结果；
- 所涉及的测试用例的覆盖度；
- 使用Git情况。

# 提交方式

- **请遵循实验报告模板撰写。**

- **提交日期：第周7周日晚(4月26日 23:55)**
- **提交两个文件到学习通平台：**
  - 实验报告：命名规则“学号-Lab3-report.doc”，具体请参见模板
  - 增加了测试用例的源代码打包：命名规则“学号-Lab3-code.zip”
  - 确保GitHub上有本次实验之后的全部代码，包含所有测试用例

- **第6周和第7周的实验课上，请实验教师/TA现场检查测试执行情况。**

哈爾濱工業大學
HARBIN INSTITUTE OF TECHNOLOGY

# 结束